Loop Engineering

Designing the cycle that prompts the agent

Loop shapes · Scorekeeper · Building blocks · Long runs
Triggers · Autonomy · Self-improvement · Guardrails ·
Frameworks

Part 1: Foundations

What a loop is · Loop shapes · The scorekeeper

Part 2: Building the loop

Building blocks · Long-running loops · Triggers and schedules

Part 3: Autonomy and improvement

The autonomy ladder · Self-improving loops

Part 4: Running it safely

Failure modes and guardrails · Framework bindings

Notes: ai-engineering/04-loop-engineering

PART 1

Foundations

What a loop is · Loop shapes · The scorekeeper

The definition

One cycle, three vocabularies

Two things before anything runs

Crossing from harness into loop

Two axes map the pillar

The five required parts

The canonical skeleton

Most loop decisions are multiple choice

Pick the pattern by task shape

Build the verifier first

Five traits of a scorekeeper

The write boundary

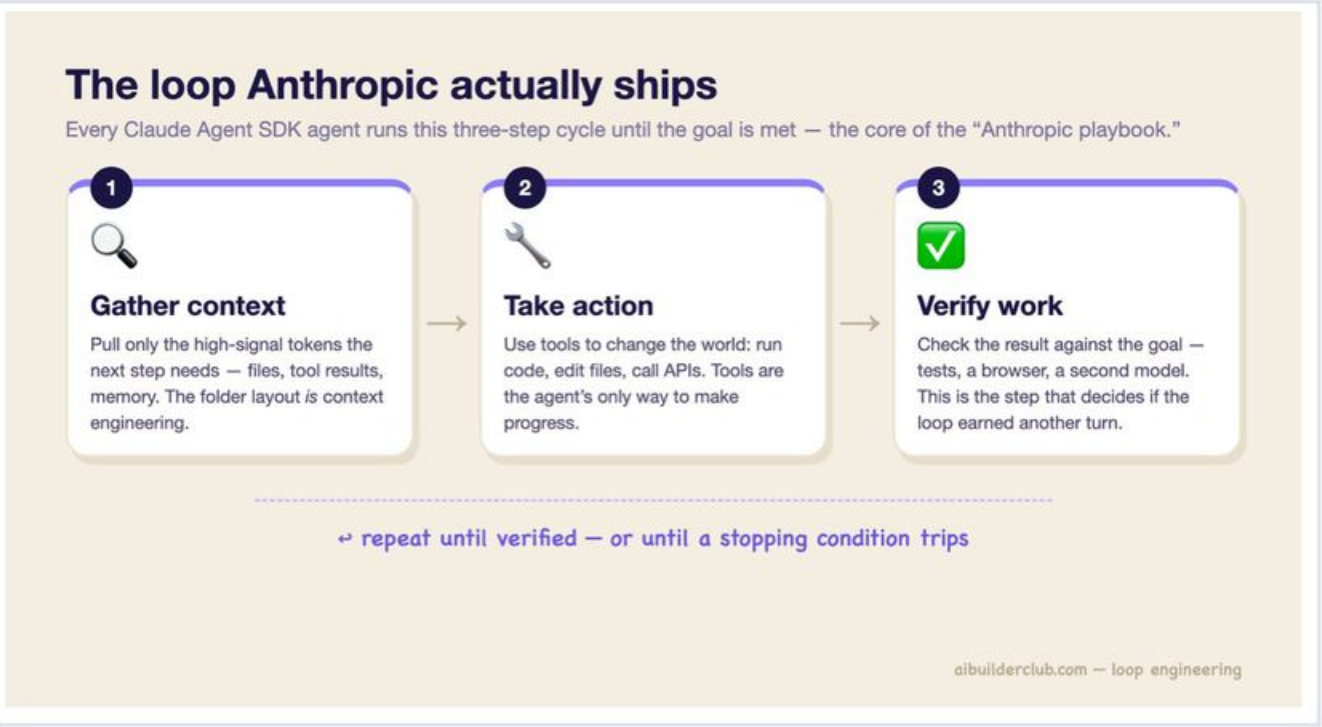
The checker never trusts the maker

The definition

> **Loop engineering is designing the cycle that keeps an agent correct and productive when nobody is watching.**

"Stop prompting your agents and start designing the loops that prompt them." — Adnan Masood

One cycle, three vocabularies



The loop Anthropic ships (AI Builder Club, 2026)

- **Classic agents:** perceive → decide → act → repeat
- **Claude Agent SDK:** gather → act → verify → repeat
- **KodeKloud:** try → test → fix → **save** → repeat
- The save step makes progress a ratchet. A bad pass costs one rollback, not the run

Source: notes/01-what-a-loop-is.md

Two things before anything runs

A clear goal

- A plain statement of what done looks like
- "Make all auth tests pass", not "improve the app"
- A testable goal is a loop invariant; a vague one is an infinite loop

Points the work

A scorekeeper

- An objective check that says whether a step helped or hurt
- Test suite, build, linter, or a rubric graded by a separate agent
- The load-bearing part, with its own note

Proves the work

Write the scorekeeper first. Then go green once: goal → one change → score → persist.

Source: notes/01-what-a-loop-is.md

Crossing from harness into loop

Inside the loop vs outside it

Harness engineering	Loop engineering
One run: tools, sandbox, hooks, verification	Many runs: what starts the next one, and when to stop
Progress file written at session end	Progress file read by the next pass to pick its own task
Generator/evaluator separation	Maker/checker as the loop's quality ceiling
Ratchet on the harness (every failure → a rule)	Ratchet on the work (score only goes up, else roll back)

> **The moment the agent's next run starts without you typing a prompt, you've crossed from harness into loop.**

Source: notes/01-what-a-loop-is.md

Two axes map the pillar

Capability levels (LangChain)

Each level wraps the one before. The second axis, autonomy, is independent of it.

Level	Loop	Automates	Note
1	Agent loop: model calls tools until done	Work	02, 04, 05
2	Verification loop: grader feeds failures back	Quality	03
3	Event-driven loop: schedules and webhooks start runs	Invocation	06
4	Hill-climbing loop: traces rewrite the harness	Improvement	08

> **Humans stay in at every level. "Automation doesn't mean removing humans from the loop."**

Source: notes/01-what-a-loop-is.md

The five required parts

A loop missing any one will hang, thrash, or quietly ship garbage.

Part	Missing it looks like
Testable goal	Loop never finishes; scope drifts
Tool set	The agent can't see consequences, so it guesses
Context management	Token overflow; it forgets what it tried
Termination logic (layered, independent)	Runs forever or stops at random
Error handling	Retries the same failed approach

> **Design the shape before the prompt. A prompt cannot fix a missing line.**

Source: notes/02-loop-shapes.md

The canonical skeleton

The model supplies two lines

PSEUDOCODE

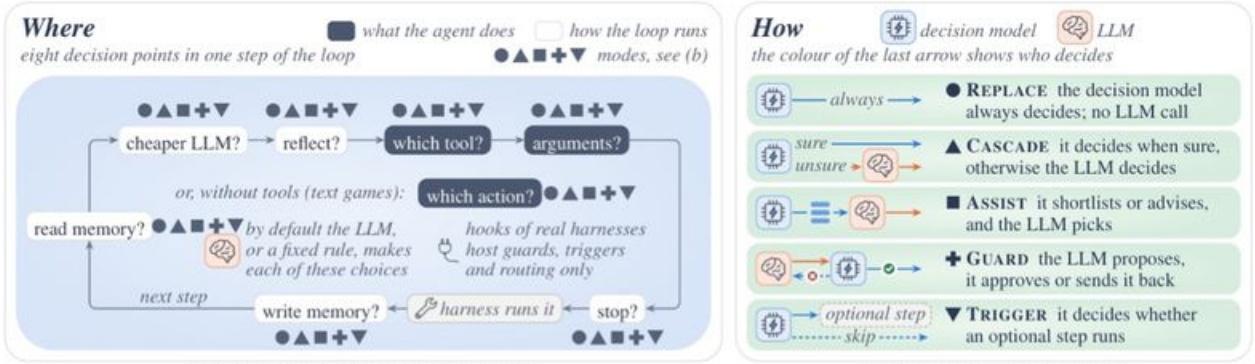
```
initialize state with goal
for step in range(MAX_STEPS):
   reason about current state      # the model
   choose a concrete action        # the model
   execute the tool
   update state with the outcome
   compact context
   if scorekeeper says goal met:
       save; return success
   if score went down: roll back
   if no progress for N steps
      or budget exhausted: escalate
escalate to human  # cap is not success
```

- "Engineering is everything wrapped around" the model (Masood)
- **Save** on success and **roll back** on a drop: the score only goes up
- The final line is an escalation, not a return value

Source: notes/02-loop-shapes.md

Most loop decisions are multiple choice

JevAgentHarness (Zhu et al., 2026)



Eight decision points in one step, and five ways to share control (Zhu et al., 2026, Fig. 3)

- A small decision model scores the options: **28%** of an LLM call's time, **2.6%** of its price
- Without a fitted threshold, delegating tool choice cut success by **19 points**
- With a per-point threshold (the cascade): **81–89%** cheaper, up to **2.5×** faster, under 3 points lost

> **A cheap component in the loop is safe only once you've measured where it is wrong.**

Source: notes/02-loop-shapes.md

Pick the pattern by task shape

Retry

Atomic task, clear pass/fail. Act → check → act again with the error.

Plan–Execute–Verify

Multi-step work where order matters. Plan once, run steps, verify each.

Explore–Narrow

Unfamiliar territory. Fan out in parallel, then converge on the best lead.

Human-in-the-loop

Ambiguity or high stakes. Pause at decision points for approval.

Vercel's order: tool use first, then reflection, then planning and multi-agent once evals exist.

Source: notes/02-loop-shapes.md

Build the verifier before you scale the generator. A generator that can't check itself just produces plausible-looking work faster.

Anthropic's web-app agent improved once it could test features like a human would — verification, not a better prompt.

Five traits of a scorekeeper

KodeKloud's four, plus one

Objective

Same work gives the same result every time.

Automatic

One command or CI trigger. No manual step.

Fast

Cheap enough to run every pass.

Clear

One unambiguous score, so the loop knows where it stands.

Hard to fake

A loop that can lower the bar will take the easiest path to green.

Defenses

Held-out checks, tamper-proof location, auditable artifacts, watch the trend.

Source: notes/03-the-scorekeeper.md

The write boundary

File	Who writes it	Why
`train.py`	Agent	The search space
`prepare.py` **(eval)**	Nobody	The verifier must not be editable by the thing it grades
`program.md`	Human	The contract

> **An agent that can edit its own evaluator does not have a verifier; it has a negotiation.**

Source: notes/03-the-scorekeeper.md

The checker never trusts the maker

MARKDOWN

```
## Your role
You are the Checker. Be strict, and
honest in both directions.
1. Read the loop contract.
2. Read the diff, not the Maker's summary.
3. Run the scorekeeper commands yourself.
## Checklist
- Boundary: any file outside it changed?
 Tests or scoring touched? → auto Fail
- Tests: would they fail if reverted?
- Score: higher, equal or lower?
## Report
Verdict: PASS / FAIL / PASS WITH NOTES
A PASS needs scorekeeper output behind it.
```

- Checker: fresh agent, **no maker history**, stricter rules
- Must be allowed to return **clean**, or it invents findings
- ADK: `RefinerAgent` / `CriticAgent`; LangChain: `RubricMiddleware`

Source: notes/03-the-scorekeeper.md

PART 2

Building the loop

Building blocks · Long-running loops · Triggers and schedules

Six parts, each fixing one failure

What each block buys a loop

The whole loop in motion

Context is a budget, not a bucket

Memory — five fields

The Ralph loop, with a ratchet

Initializer and worker

Two ways a loop runs again

Three triggers, and the cost ceilings they need

Six parts, each fixing one failure

Part	Fixes this failure
Automations	You are still the one pressing go
Worktrees	A bad try breaks main; parallel agents collide
Skills	The agent re-guesses your conventions every pass
Connectors / plugins	The loop can only edit local files
Sub-agents	Main context floods; the maker grades itself
Memory	Each restart forgets what was done

> **Add a part when you see the failure it fixes, not before.**

Source: notes/04-building-blocks.md

What each block buys a loop

Worktrees

Disposable, concurrent experiments. `git worktree add ../try -b try`; a failed try costs one `remove`.

Skills

Steps, not essays, loaded only when the task matches. Skipping them is intent debt.

Connectors

From "here's the fix" to "PR opened, ticket linked". Least privilege: read-only by default.

Sub-agents

Fresh context, one job, distilled return. Explorer → implementer → verifier.

Source: notes/04-building-blocks.md

The whole loop in motion

One pass, KodeKloud's sequence

one pass

1. **Goal + scorekeeper** — required before anything runs
2. **Read memory, make a worktree** — where did I leave off; a safe place to try
3. **Load the skill, make ONE change** — the steps for this kind of job
4. **Score it** — the scorekeeper, not the agent's own claim
5. **Save or roll back, update memory** — the ratchet, then the next pass starts

Source: notes/04-building-blocks.md

Context is a budget, not a bucket

A loop that runs for hours must forget on purpose and remember on disk. Progress lives in files, not in the context window.

Context rot, context anxiety and session end all have the same fix.

Memory — five fields

Read at the start of every pass

Field	Example
Goal	`All tests in tests/auth/ pass`
Last score	`41/47 passing`
What was tried	`Mocking the clock in test_expiry: no change`
What is next	`Check token refresh path in auth/session.py`
Waiting for a human	`Approve dropping the legacy cookie`

> **"What was tried" earns its place. Without it, a fresh pass reruns the attempt that failed an hour ago.**

Source: notes/05-long-running-loops.md

The Ralph loop, with a ratchet

BASH

```
best=$(score)
for ((i = 1; i <= MAX_ITERS; i++)); do
 past_deadline && exit 2        # clock
 $AGENT_CMD < "$PROMPT_FILE"    # fresh ctx
 now=$(score)
 if worse; then                 # undo
   git reset --hard; ((stall++))
 elif better; then              # keep
   git commit -qam "pass $i"
   best=$now; stall=0
 else ((stall++)); fi
 done_check && exit 0           # success
 ((stall >= STALL_LIMIT)) && exit 3
done
exit 1                           # cap != success
```

- Iteration cap, wall-clock cap, a **done check** run by the scorekeeper, a stall check
- Exit codes tell success (0) from cap (1), clock (2), stall (3)
- Harnesses now ship it: stop hooks, `/goal`

Source: notes/05-long-running-loops.md

Initializer and worker

Initializer (runs once)

- Turns the task into files
- Feature list, progress log, scaffolding, `init` script
- Commits them

The plan on disk

Worker (every session)

- Reads the progress log and git history
- Picks **one** unfinished item, implements and tests it
- Commits, updates the log, exits cleanly

One item per session, leave the repo clean

Loops lean toward reset over compaction: state is already on disk, so clear the window and hand off through files.

Source: notes/05-long-running-loops.md

Two ways a loop runs again

Run-until-done

- Pass after pass, immediately
- Stops on the finish signal or a cap
- Needs a precise done

`/goal`

Timer

- On an interval or cron
- Never stops by itself; each run is short
- Must be cheap when there's nothing to do

`/loop 5m`, `/schedule`

Does this thing have an end? Yes → /goal. No, it's ongoing watching → /loop.

Source: notes/06-triggers-and-schedules.md

Three triggers, and the cost ceilings they need

Trigger	Fires on	Example
Manual	You	Interactive work you're watching
Event	Sentry error, ticket, failing CI, Slack	Investigate the alert and open a PR
Schedule	The clock	Nightly test-babysitting; 30-minute migration cycles

> **Prefer events to polling. Caps per run and per day, a no-op path that exits before the model, alerts on spend.**

Source: notes/06-triggers-and-schedules.md

PART 3

Autonomy and improvement

The autonomy ladder · Self-improving loops

The autonomy ladder

The evolve loop

The continual-evolution loop

Four edit targets, and the anti-busywork rule

Verify before you learn

Where the lesson goes

Self-modification is a release process

The gate runs outside the model

Plasticity costs stability

autoresearch: one file, one GPU, one metric

At the frontier — recursive self-improvement

The autonomy ladder

Climb one rung at a time

more handed off

1. **1 — Turn-based** — hand off tool approval within a turn. Where you learn how tasks fail
2. **2 — Goal-based (`/goal`)** — hand off the stop condition. Needs a measurable end state
3. **3 — Time-based (`/loop`)** — hand off the trigger to a clock. Session-scoped, your permissions
4. **4 — Proactive (`/schedule`)** — hand off trigger and session. Needs a verifier you trust unattended

> **Earn each handoff with a verifier. Skipping rungs is how you get a doom loop.**

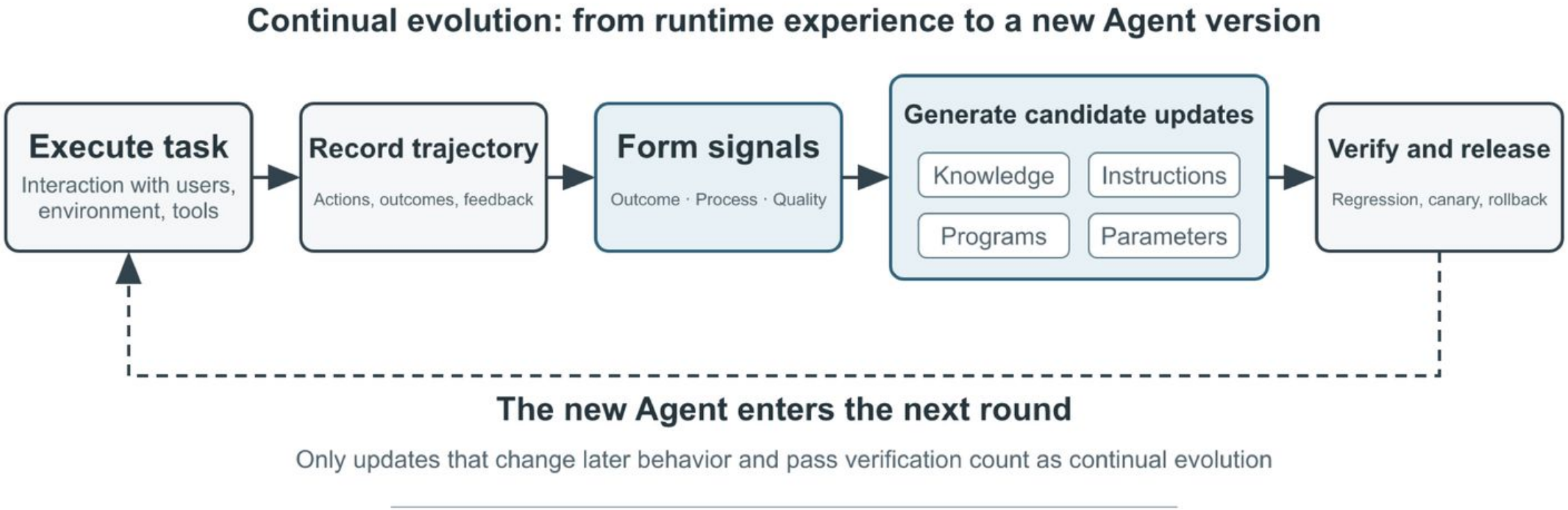
Source: notes/07-autonomy-ladder.md

The evolve loop

> **An evolve loop is a loop with a second, slower loop pointed at it. The fast loop does the work. The slow loop edits the fast loop.**

It doesn't retrain weights; it rewrites files. Contract, state, triggers and procedure improve while the model stays fixed.

The continual-evolution loop



Execute, record, form signals, generate candidate updates, verify, release (Li, 2026, Fig. 9-1, Apache-2.0)

Source: notes/08-self-improving-loops.md

Four edit targets, and the anti-busywork rule

Target	Symptom that triggers it
Contract	Loop "succeeds" on runs you'd call failures
State	Agent acts on facts that expired
Trigger logic	Wakes constantly, finds nothing to do
SOP steps	Same manual correction, every run

> **Agents default to doing something. Make "no change needed" an explicit, successful outcome.**

Source: notes/08-self-improving-loops.md

Verify before you learn

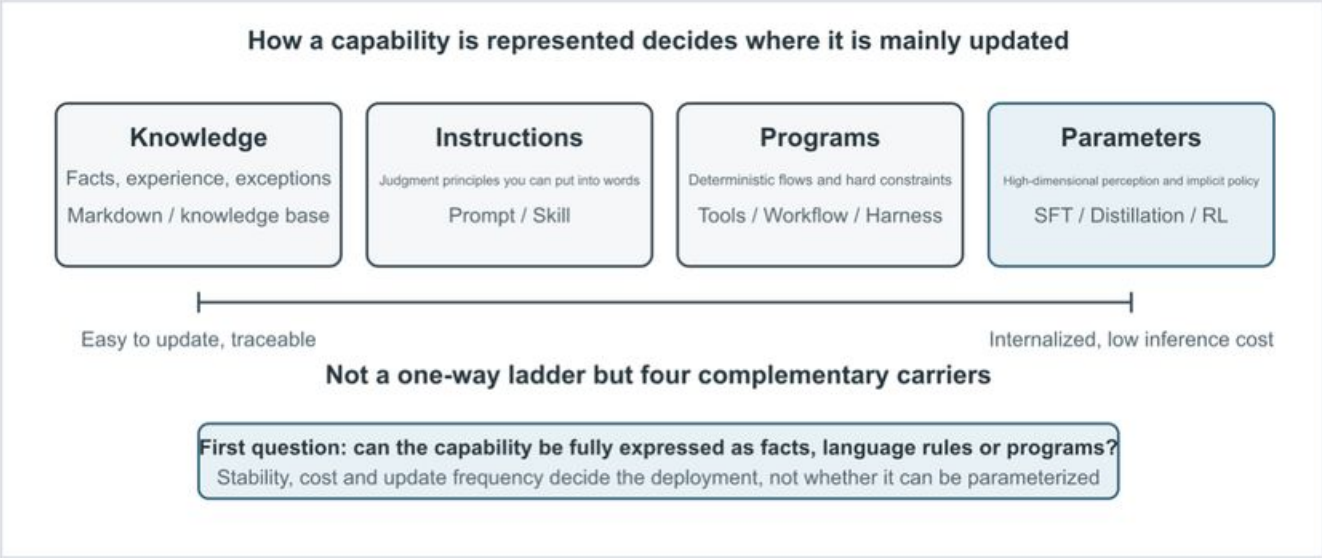
PYTHON

```
def _promise_action(self, traj):
    ok = {c["name"]
          for c in traj["tool_calls"]
          if c["result"].get("success")}
    missing = [p for p in traj["promises"]
               if p["required_tool"] not in ok]
    if missing:           # said it, never did it
        return DimensionResult(
            "promise_action_consistency",
            "process_rules", FAIL, 0.0,
            [f"turn {p['turn']}: {p['text']}"
             for p in missing],  # evidence
            confidence=1.0)
```

- Three layers: **outcome** and **process** in code, **quality** by rubric
- False promise and privacy leak both average **0.571** but need different fixes
- Structured, not scalar. A single 0.57 points nowhere

Source: notes/08-self-improving-loops.md

Where the lesson goes



Knowledge docs, prompts and Skills, programs and harness, parameters (Li, 2026, Fig. 9-3, Apache-2.0)

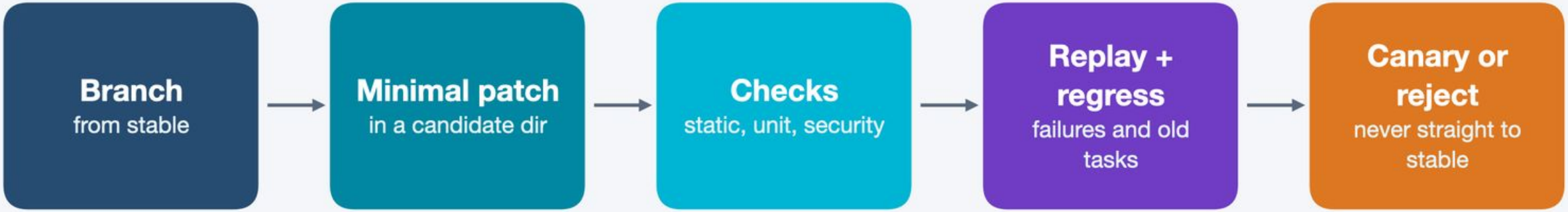
- Starts as an **experience doc**, from comparison not summary
- Promoted to a **Skill** rule after repeated validation
- Compiled into **code** if it needs no language understanding
- **Weights** only for broad, implicit judgment

> **Choose the smallest target that is easiest to verify and roll back.**

Source: notes/08-self-improving-loops.md

Self-modification is a release process

Never overwrite stable



- **Falsifiable change contract:** root cause, expected gain, possible regression, tests for both
- **Bounded candidate space:** failures, successes to preserve, rejected attempts
- Safety is not self-modifiable: validators, thresholds and audit logs stay out of reach

Source: notes/08-self-improving-loops.md

The gate runs outside the model

Li, Exp. 9-6

PYTHON

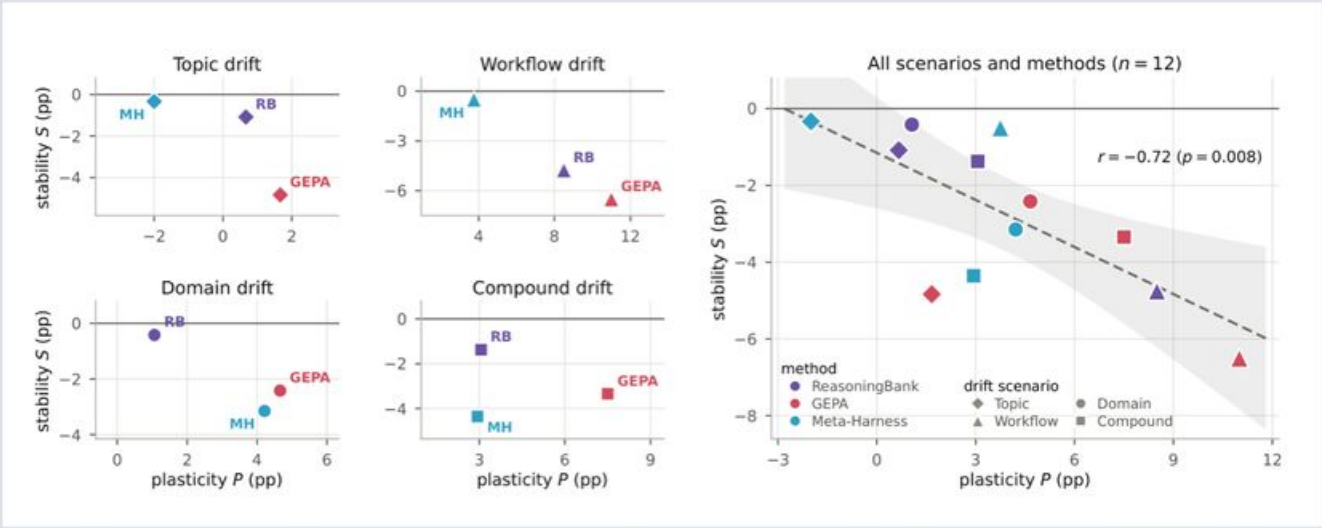
```
retry = ns["should_retry"]
checks["failure_replay"] = all(
   not retry(t["error_code"],
             t["retryable"], 0)
   for t in failures)
checks["old_task_regression"] = all((
   retry("TEMPORARY_TIMEOUT", True, 2),
   not retry("TEMPORARY_TIMEOUT", True, 3),
))
decision = ("release_to_canary"
            if all(checks.values())
            else "reject_candidate")
```

- Trigger: **≥ 2** failure trajectories with the same tool and error code
- Patch: stop retrying `retryable=false` errors
- Swap in an LLM patch writer: a plausible patch that breaks timeouts still gets **reject**

Source: notes/08-self-improving-loops.md

Plasticity costs stability

Chen et al., 2026



Gain on new tasks vs loss on old ones, four drift scenarios (Chen et al., 2026, Fig. 2)

- Across 12 runs: the more a method learned, the more it forgot (**r = –0.72**)
- Global edits to one shared prompt: a family's share fell **100% → 14.7%**
- Opposite workflows → the policy rewritten into its opposite
- Measure separately: validity, **activation**, **adherence**, held-out gain

Source: notes/08-self-improving-loops.md

autoresearch: one file, one GPU, one metric

The loop

- Agent edits `train.py`; nobody edits `prepare.py`
- Fixed **5 minutes** of training per try, so results compare
- Better → keep and log to `results.tsv`; worse → `git reset HEAD~1`
- The method lives in English, in `program.md`

Karpathy, 2026-03-06, ~630 lines

The result

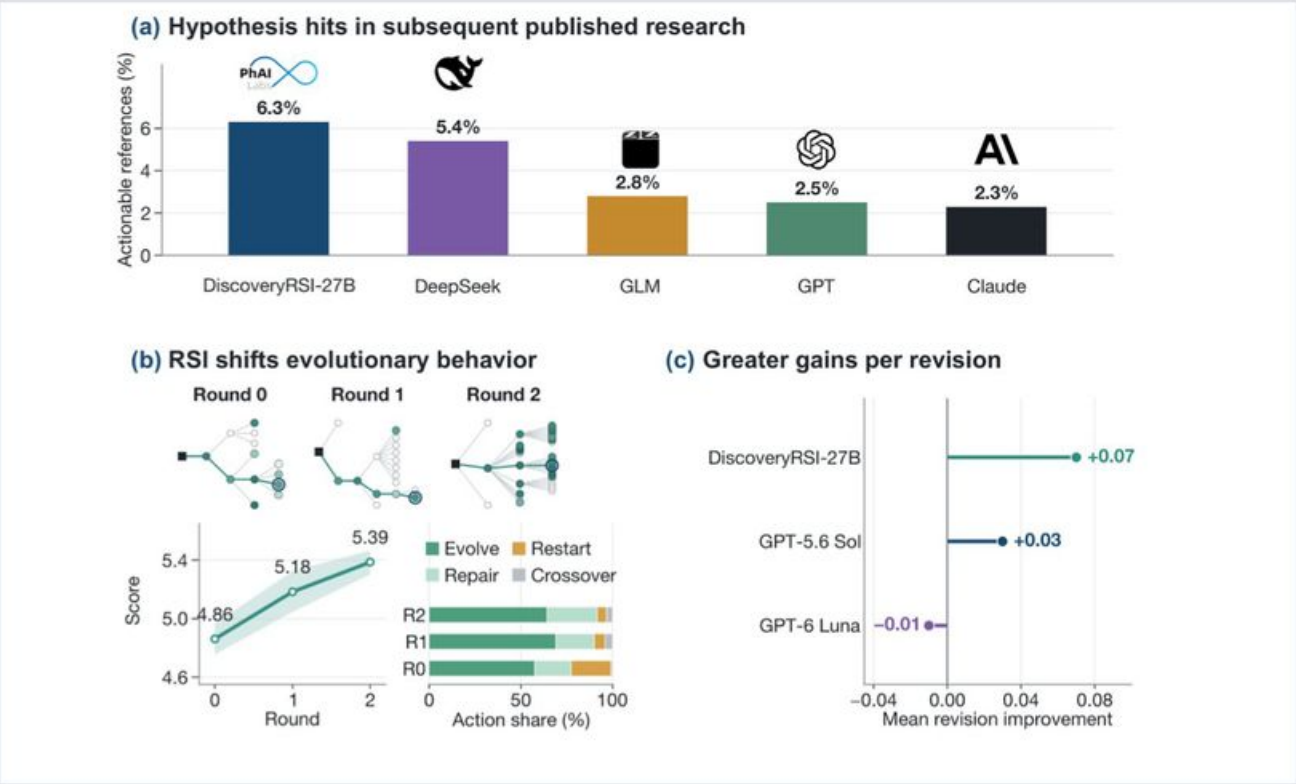
- 700 experiments over two days → **20** real improvements
- GPT-2 training **2.02 → 1.80 h** (11%)
- Hit rate **under 3%**, which is fine when attempts are cheap
- Bilevel version: outer loop writes search code, **5×** gain, same model

"If you have an objective metric, you are the bottleneck."

Source: notes/08-self-improving-loops.md

At the frontier — recursive self-improvement

DiscoveryRSI (Zhong et al., 2026)



Hits in later research, quality over two RSI rounds, human-rated gain (Zhong et al., 2026, Fig. 1)

- Task horizon **doubling every ~4 months**; >80% of Anthropic's merged code by Claude
- Quality is an automated rubric; best hit rate vs later papers **6.3%**
- Open-ended work: implementation drift, over-optimism, missing taste

> **The limit becomes verification capacity, not generation capacity.**

Source: notes/08-self-improving-loops.md

PART 4

Running it safely

Failure modes and guardrails · Framework bindings

- The traps
- The four silent costs
- Layered exits
- Guardrails for self-modifying loops
- The same loop parts in three frameworks
- ADK: the critic/refiner loop

The traps

Failure modes (KodeKloud, Masood)

Trap	Symptom	Fix
No scorekeeper	Results inconsistent or meaningless	Write it first; objective and hard to game
Goal too big	Never finishes; scope drifts	Cut to one small testable piece
Cheating the tests	Green score, wrong output	Protect the scorekeeper; held-out checks
Thrashing	Lots of change, no improvement	Smaller goal, smaller changes
Context rot	Memory degrades as the transcript grows	Summarize and prune inside the loop
Unsafe autonomy	Risky commands or sweeping edits	Narrow write scope; approval gates

Source: notes/09-failure-modes-and-guardrails.md

The four silent costs

Osmani: they don't show up in traces

Verification debt

"Looks fine" stands in for "confirmed". Make stop conditions machine-checkable.

Comprehension rot

Code ships faster than you understand it. Fast loops require fast reading.

Cognitive surrender

"The comfortable posture is the dangerous one." You take what it returns without an opinion.

Token blowout

Context grows roughly quadratically. Compaction or reset from the first loop.

> **"Build it like someone who intends to stay the engineer, not just the person who presses go."**

Source: notes/09-failure-modes-and-guardrails.md

Layered exits

Several independent ways to stop

Exit	Fires when	Outcome
Success	Scorekeeper confirms the goal	Save, report, stop
Iteration cap	N passes reached	Escalate. The cap isn't success
Budget cap	Tokens, dollars or wall-clock spent	Escalate with state saved
Stall detector	Score flat for K passes, or same action repeated	Change strategy once, then escalate
Human escalation	Blocker, irreversible action, judgment call	Pause and hand over full context

> **The two ratchet rules: the score only goes up, and fix breaks before adding more.**

Source: notes/09-failure-modes-and-guardrails.md

Guardrails for self-modifying loops

Boundary	Rule
Evidence ≠ instructions	Web pages and tool output are untrusted. Summarize, version as a PR, review with another model family
Candidate ≠ production	New knowledge, prompts, Skills, code start where no traffic is served. Sandbox, scan, test, then canary
Safety is not self-modifiable	Validators, tests, thresholds, audit logs and backups are out of the agent's reach

> **The third rule is the write boundary, extended to the release gate.**

Source: notes/09-failure-modes-and-guardrails.md

The same loop parts in three frameworks

Loop part	Google ADK	Vercel AI SDK	LangGraph
Agent loop	`LoopAgent`	`ToolLoopAgent`	`create_agent`
Checker	critic sub-agent	your check in a tool	`RubricMiddleware`
Early exit	`escalate = True`	`stopWhen`	edge to `END`
Hard cap	`max_iterations`	`stepCountIs(n)`	`recursion_limit`
Durability	session state	`WorkflowAgent`	checkpointer
Trace	events	`result.steps`	LangSmith

> **Look for where each framework puts the goal, the scorekeeper and the stop rules.**

Source: notes/10-framework-bindings.md

ADK: the critic/refiner loop

PYTHON

```python
def exit_loop(tool_context: ToolContext):
   tool_context.actions.escalate = True
   return {}

critic = LlmAgent(name="CriticAgent",
   instruction="Grade {draft} ...",
   output_key="critique")
refiner = LlmAgent(name="RefinerAgent",
   instruction="If {critique} is clean, "
               "call exit_loop. Else rewrite.",
   tools=[exit_loop], output_key="draft")
refine = LoopAgent(name="Refine",
   sub_agents=[critic, refiner],
   max_iterations=5)       # backstop
```

- **Stopping is not automatic:** a sub-agent must set `escalate`
- `output_key` writes state; `{draft}` reads it back
- `max_iterations` is the backstop, **not** the main exit

Rules of thumb

The pillar on one slide

Wrap-up

Foundation

Goal and scorekeeper before anything runs. Go green once, then add parts.

Scorekeeper

Deterministic first, out of the loop's write scope, some checks held out. The checker can say clean.

Ratchet

The score only goes up. One change per pass; save or roll back.

Long runs

State on disk, not in context. One item per session, leave the repo clean.

Autonomy

Climb one rung at a time, each earned with a verifier you trust.

Stopping

Layered, independent exits. A cap is an escalation, never a success.

Source: README.md

Where to go next

Wrap-up

- **05 — Graph engineering:** loops composed into nodes, edges and multi-agent topologies
- **06 — Eval:** the scorekeeper as an eval artifact; graders, judges, Goodhart
- **Labs:** upstream ai-agent-book chapter9 trajectory-verifier and self-modifying-agent (worked through in 08)
- **Copy from the notes:** loop contract (09), loop state and ralph.sh (05), maker and checker prompts (03)
- **Read:** Li, AI Agents in Depth ch. 9; Karpathy's autoresearch; Osmani and Masood

The one idea to keep

The model supplies two lines of the loop. Everything else — the goal, the scorekeeper, the stop rules — is yours to design.

Source: README.md